

Tez Konusu Önerisi: OpenAI Whisper ASR Modelinin Nicemleme Yöntemleriyle Uç Nokta Cihazlarda Yüksek Performanslı Dağıtımı Üzerine Kapsamlı Literatür Taraması

1. Giriş ve Tez Konusunun Çerçevesi

1.1. Otomatik Konuşma Tanıma (ASR) Teknolojilerinde Paradigm Değişimi

Otomatik Konuşma Tanıma (ASR) sistemleri, son on yılda geleneksel modüler yaklaşımlardan Uçtan Uca (E2E) sinir ağı tabanlı çözümlere doğru kritik bir paradigim kayması yaşamıştır. Geleneksel sistemler akustik, telaffuz ve dil modellerini ayrı ayrı gerektirirken, E2E modeller akustik özelliklerden metne doğrudan bir eşleme öğrenerek sistemi basitleştirir ve performansı artırır (1). Bu modern mimariler, Connectionist Temporal Classification (CTC), Recurrent Neural Network Transducers (RNN-T) ve özellikle dikkat mekanizmalarını kullanan Transformatör (Transformer) ve Conformer modelleri etrafında şekillenmiştir (2).

Bu teknolojik ilerlemeyle birlikte, veri paradigmasında da büyük bir dönüşüm yaşanmıştır. İlk dönemlerdeki denetimli öğrenme yaklaşımlarının aksine, alan artık devasa miktarda etiketsiz ses verisinden güç alan Öz-Denetimli Öğrenme (SSL) çerçevelerine (örneğin wav2vec 2.0) yönelmiştir. Eş zamanlı olarak, OpenAI tarafından sunulan Whisper modeli, Büyük Ölçekli Zayıf Denetim (Large-Scale Weak Supervision) trendini zirveye taşımıştır. Bu yaklaşım, modeli 680.000 saatin üzerinde, çok dilli ve çok görevli web ölçekli veri ile eğiterek, herhangi bir ince ayar gerektirmeden standart karşılaştırmalarda yüksek sağlamlık (robustness) ve sıfır-atış (zero-shot) transfer yeteneği sergilemiştir (3).

1.2. OpenAI Whisper Modelinin ASR Alanındaki Temel Katkısı ve Dağıtım Zorlukları

OpenAI Whisper, otomatik konuşma tanıma, konuşma çevirisi ve dil tanımlama gibi çoklu görevleri gerçekleştirebilen, güncel teknoloji harikası bir model olarak kabul edilmektedir (5). Modelin başarısının ardındaki itici güç, eğitildiği verinin hem nicelik (680.000 saat) hem de çeşitlilik (İngilizce dışında 96 dil) açısından büyüklüğüdür (3). Bu sağlam yaklaşım, Whisper'ı hem akademik araştırmalarda hem de ticari uygulamalarda önde gelen ASR çözümlerinden biri

yapmıştır (6).

Ancak, Whisper'ın yüksek doğruluğu, önemli bir hesaplama yüküyle birlikte gelmektedir. Model varyantlarının (tiny, base, small, medium, large) yüz milyonlarca parametreye sahip olması, özellikle mobil cihazlar, IoT ve gömülü sistemler gibi kaynak kısıtlı ortamlarda (resource-constrained devices) dağıtımı zorlaştırmaktadır (6). Büyük model varyantları, artan gecikme (latency) ve yüksek hesaplama talepleri sergileyerek, gerçek zamanlı ASR hizmetlerinin sunulmasını engeller (6). Bu kısıtlamalar, model sıkıştırma ve optimizasyon tekniklerinin zorunlu hale gelmesine yol açmıştır.

Tez çalışmasının temel zorunluluğu, Whisper'ın elde ettiği yüksek transkripsiyon doğruluğunu korurken, uç cihazlarda dağıtımı mümkün kılacak şekilde sistemin gerçek zamanlı performansını optimize etmektir. Bu optimizasyonun değerlendirilmesi için kilit metrikler kullanılır. Doğruluk için Word Error Rate (WER) ve Character Error Rate (CER) kullanılırken, sistemin canlı ses akışına ayak uydurabilmesi için Real-Time Factor (RTF) hayati önem taşır. Bir sistemin gerçek zamanlı çalışabilmesi için RTF değerinin **1.0** 'ın altında olması gerekmektedir (7).

1.3. Paradigma Çatışması ve Gecikme Metriklerinin Önemi

Whisper, sadece bir ASR sistemi değil, aynı zamanda ses verisi üzerinde çalışan bir 'Audio Large Language Model (LLM)' olarak da tanımlanmaktadır (6). Bu çifte doğa, LLM'lerin doğasındaki büyük ölçekli yapıyı, ASR'ın acil düşük gecikme gereksinimiyle çatıştırmaktadır. Yüksek doğruluk için büyük modeller gerekliliği, düşük kaynaklı cihazlarda infazın imkansız hale gelmesi anlamına gelir.

Bu durum, nicemleme (quantization) tekniklerini yalnızca bir verimlilik aracı olmaktan çıkarıp, Whisper'ın geniş erişilebilirliğini sağlayan temel bir gereklilik haline getirir (6). Ayrıca, büyük modellerin değerlendirilmesinde sadece WER'e odaklanmak yetersizdir. Yapılan araştırmalar, Whisper'ın transkripsiyonlarının yaklaşık %1'inin tamamen halüsinasyon içerikli cümle veya ifadeler barındırdığını göstermektedir. Bu halüsinasyonların %38'inin ise yanlışlıklar veya şiddet gibi zararlı içerikler içerdiği tespit edilmiştir (6). Bu bulgu, tezin nicemlenmiş modellerde sadece sayısal hata oranını değil, aynı zamanda modelin semantik sağlamlığını (yani halüsinasyon riskini) da incelemesi gerektiğini göstermektedir.

2. Uç Nokta ASR Dağıtımı İçin Nicemleme (Quantization) Teori ve Gereklilikleri

2.1. Nicemleme Mekanizmasının Temelleri ve Amaca Yönelik Kullanımı

Nicemleme, bir sinir ağı (NN) veya büyük dil modelinin (LLM) ağırlıklarını ve/veya aktivasyonlarını, genellikle standart **FP32** (32-bit kayan nokta) hassasiyetinden **INT8** (8-bit tam sayı) veya daha düşük kesinlikli bir formata dönüştürme sürecidir (6). Bu dönüşüm, modelin

bellek ayak izini büyük ölçüde azaltır ve düşük hassasiyetli hesaplamalar sayesinde donanım hızlandırmayı mümkün kılar (6).

Whisper modeli üzerinde yapılan ampirik çalışmalar, nicemlemenin bu faydalarını net bir şekilde göstermektedir. Örneğin, **INT4**, **INT5** ve **INT8** nicemleme yöntemleri kullanılarak yapılan bir analiz, nicemlemenin model boyutunu %45 oranında azalttığını ve gecikmeyi %19 oranında düşürdüğünü, bunu yaparken transkripsiyon doğruluğunu (WER) büyük ölçüde koruduğunu ortaya koymuştur (6). Bu bulgular, nicemlenmiş Whisper modellerinin, bellek ayak izini azaltarak ve yüksek doğruluğu koruyarak, gerçek zamanlı transkripsiyon hizmetlerini mobil cihazlar, IoT ve gömülü sistemler gibi kaynak kısıtlı ortamlarda mümkün kılacağını göstermektedir.⁶

2.2. Nicemleme Stratejileri: Dinamik, Statik ve Bit Kesinliği

Uç nokta dağıtımı için en yaygın kullanılan teknik, yeniden eğitim (retraining) gerektirmeyen **Eğitim Sonrası Nicemleme (Post-Training Quantization - PTQ)** yöntemidir (10). PTQ, modelin hızlı bir şekilde sıkıştırılmasını ve düşük optimizasyon maliyetiyle dağıtılmasını sağlar. PTQ, genellikle **Dinamik** ve **Statik** olmak üzere iki ana şemaya ayrılır.

- **Dinamik Nicemleme (Dynamic Quantization):** Bu şema, model ağırlıklarını önceden nicemlerken, aktivasyon aralıklarını infaz sırasında dinamik olarak belirler. Whisper-small üzerinde yapılan kapsamlı bir değerlendirme, dinamik nicemlemenin statik nicemlemeyi tutarlı bir şekilde geride bıraktığını göstermiştir (12). Transformatör mimarisi için, dinamik nicemleme hem doğruluk hem de çıkarım hızında üstün performans sergileyerek, Whisper-small için pratik tek seçenek haline gelmiştir.
- **Statik Nicemleme (Static Quantization):** Statik nicemleme, hem ağırlıkları hem de aktivasyon aralıklarını önceden kalibre etmeyi amaçlar. Ancak, Whisper'ın Transformatör mimarisindeki LayerNorm ve Softmax gibi işlemler, statik nicemlemeye uygun değildir. Bu durum, statik nicemlemenin hız kazanımlarını engellemesine ve performansının önemli ölçüde düşmesine neden olmuştur (12).

Bit kesinliği seçimi, sıkıştırma ve doğruluk arasındaki temel denge noktasını oluşturur. **INT8** (8-bit), cihazlar arasında en güvenilir kesinlik ayarı olduğunu kanıtlamıştır (10). Daha agresif formatlar, örneğin **INT4** veya **INT3**, model boyutunda %71'e kadar maksimum sıkıştırmaya olanak tanır (12). Ancak, bu agresif sıkıştırma, özellikle gürültülü veya zorlu akustik koşullar altında modelin sağlamlığını ve doğruluğunu tehlikeye atmaktadır. Analizler, **INT8** kesinliğin altına düşüldüğünde modelin kararlılığının azaldığını göstermektedir.

2.3. Pratik Zorunluluk ve Algoritmik Detaylar

Nicemleme ihtiyacı, akademik bir merakın ötesinde, ticari ve erişilebilirlik açısından pratik bir zorunluluktur. Düşük RTF hedefine ulaşmak ve küçük bellek kısıtlamalarını karşılamak, mobil ve IoT uygulamaları için temeldir. Nicemleme, yüksek doğruluklu ASR sistemlerinin bu donanım kısıtlamalarını aşmasını sağlayarak, sesli etkileşim teknolojilerinin yaygınlaşmasını mümkün

kılmaktadır.

Nicemlemede sadece bit sayısını düşürmek değil, aynı zamanda kullanılan nicemleme algoritmasının kalitesi de kritik öneme sahiptir. GGML/GGUF çerçevelerinde kullanılan $Q4_{KM}$ gibi özelleşmiş nicemleme tiplerinin başarısının ardındaki algoritmik detay, ölçek faktörünün belirlenmesinde yatmaktadır. Geleneksel nicemleme yaklaşımları, ölçek faktörünü tensörün maksimum mutlak değerine göre seçer. Ancak bu, suboptimal sonuçlara yol açabilir (14). Daha gelişmiş nicemleme yöntemleri, $Q4_{KM}$ 'de olduğu gibi K-means kümelemesi kullanarak (15) veya Hata Kareleri Ortalaması (RMS) gibi bir hatayı minimize eden bir doğrusal regresyon yaklaşımıyla optimal ölçek faktörünü belirlemeyi önerir(14). Bu hassas ölçek faktörü optimizasyonu, düşük bit kesinliklerinde bile (örneğin 4-bit) kabul edilebilir doğruluk düzeylerinin korunmasını sağlar.

3. Nicemleme Yöntemlerinin Karşılaştırmalı Analizi: Kütüphane, Şema ve Donanım Etkileşimi

3.1. Kütüphane Eko-Sistemi ve Optimal WER/RTF Ticaret Noktaları

Whisper-small üzerinde gerçekleştirilen kapsamlı akademik değerlendirmeler, Post-Training Quantization (PTQ) yöntemlerinin etkinliğini dört ana kütüphane üzerinden karşılaştırmıştır: PyTorch, Optimum-Quanto, HQQ ve bitsandbytes (BNB) (10). Bu karşılaştırmalar, nicemleme şemasının ve hedef donanımın (CPU veya GPU) optimal performans üzerindeki kritik etkileşimini ortaya koymuştur.

Optimal GPU Çözümü: Quanto kütüphanesi kullanılarak uygulanan **dinamik int8 nicemlemesi**, GPU üzerinde en sağlam ve yüksek doğruluklu sonucu vermiştir. Bu yöntem, model boyutunu %57 oranında azaltırken, gürültülü test verilerinde (LibriSpeech test-other) WER'yi temel **FP32** modelinin bile altına çekmeyi başarmıştır ($WER_{other} = vs11.88$). Bu durum, nicemlemenin hafif bir düzenleme (regularization) etkisi yaratabileceğini göstermektedir.

Optimal CPU Çözümü: CPU tabanlı infazda ise, PyTorch'un dinamik **int8** uygulaması, model boyutunu %57 küçültmenin yanı sıra en hızlı çıkarımı (RTF **0.077**) sağlamıştır. Temel **FP32** modeline kıyasla çıkarım süresi %57.1 oranında iyileşmiştir (12).

Bu sonuçlar, donanıma özgü bir ticaret noktası olduğunu göstermektedir: CPU optimizasyonunda (PyTorch), daha hızlı çıkarım (düşük RTF) ana hedefken, GPU optimizasyonunda (Quanto), yüksek sağlamlık ve doğruluk daha ön plandadır. Uç cihaz dağıtımında (çoğunlukla CPU merkezli platformlar), düşük RTF kritik olduğundan, PyTorch'un basit ama hızlı şeması cazip olabilir. Ancak daha yüksek doğruluk gerektiren gürültülü ortamlarda (Quantoda görüldüğü gibi), GPU destekli daha karmaşık bir şema tercih edilebilir.

Table 3.1, bu kütüphanelerin Whisper-small üzerindeki temel performans karşılaştırmasını

sunmaktadır (12):

Table 3.1: Nicemleme Kütüphanelerinin Whisper-small Performans Karşılaştırması (LibriSpeech Test Verisi)

Kütüphane/Yöntem	Nicemleme Şeması	Bit-Genişliği	Model Boyutu Küçültme	WER (Test-Clean)	WER (Test-Other)	Çıkarma Hızı (RTF)	Temel Çıkarma
CPU Baseline	FP32	32-bit	–	3.48	11.88	0.121	Referans
PyTorch Dynamic	Per-tensor, Asimetrik	int8	57%	3.72	13.67	0.077	En Hızlı CPU Çıkarması
Optimum-Quantum Dynamic	Per-channel, Simetrik	int8	57%	3.41	10.65	0.008	En İyi Doğruluk/Robustness (GPU)
HQQ Dynamic	Dinamik	int4	69%	3.52	14.09	0.155	Yüksek Sıkıştırma
HQQ Dynamic	Dinamik	int3	71%	4.11	12.93	0.019	Maksimum Sıkıştırma

c							ma
---	--	--	--	--	--	--	----

3.2. Nicemleme Granüleritesi ve Şemasının Önemi

Nicemleme şemasının granüleritesi (incelik derecesi) performansı doğrudan etkiler. **Per-tensor** nicemleme, tüm tensör için tek bir ölçek faktörü kullanırken, **per-channel** nicemleme, her bir çıktı kanalı için ayrı ölçek faktörleri belirler. Quanto dynamic **int8**'in GPU'da üstün sağlamlık sağlamasının anahtarı, Whisper'ın sıfıra yakın ağırlık dağılımlarıyla uyumlu olan **simetrik, per-channel** nicemleme şemasını kullanmasıdır. Per-channel yaklaşımı, kanal bazında ince taneli varyasyonu daha iyi koruyarak daha yüksek doğruluk sağlar. Öte yandan, PyTorch'un kullandığı daha basit **per-tensor asimetrik** şema, hesaplamayı basitleştirmesi nedeniyle CPU'da en yüksek hıza ulaşmıştır.

3.3. Donanım Odaklı Seçim ve Hızlı PTQ Alternatifleri

Daha agresif düşük bit formatları (örneğin **nf4**, **INT3**) maksimum model sıkıştırmaı mümkün kılarsa da, bu formatlar tutarlı bir şekilde sağlamlığı düşürmektedir. HQQ dynamic **int3**, model boyutunu %71 oranında küçülterek en küçük boyuta ulaşmıştır, ancak bu, gürültülü verilerde yüksek hata oranları pahasına gerçekleşmiştir (12). Bu durum, 8-bit altındaki kesinliğin gerçek dünya ASR uygulamalarında güvenilirlik sorunları yaratabileceğini göstermektedir.

Literatür, geleneksel PTQ yöntemlerine kıyasla daha hızlı ve verimli sıkıştırma sağlayan yeni yaklaşımların ortaya çıktığını da göstermektedir. Örneğin, GenPTQ gibi yöntemler, Whisper ve Conformer modellerine uygulandığında, ortalama **\$2.5\$ – bit** kesinlikle %89.1'e varan model boyutu küçültmesini sadece 15 saniyede tamamlayabilmektedir (11). Bu, tezin sadece standart kütüphanelerle sınırlı kalmaması, aynı zamanda en yeni ultra-hızlı sıkıştırma tekniklerini de literatürde karşılaştırarak, gelecekteki dağıtım stratejilerine yön vermesi gerektiğini işaret etmektedir.

4. İnfaz(Execution) Optimizasyon Çerçeveleri: GGML, GGUF ve whisper.cpp Mimarisi

4.1. GGUF Formatının Mimarisi ve Uç Dağıtımdaki Rolü

Uç nokta infazının verimli bir şekilde gerçekleştirilmesi için, nicemlenmiş modellerin özel bir dosya formatında paketlenmesi gerekmektedir. GPT-Generated Unified Format (GGUF), bu gereksinimi karşılamak üzere geliştirilmiş, özellikle tüketici düzeyindeki donanımlarda iyi performans göstermek üzere tasarlanmış ikili bir dosya formatıdır (16).

GGUF, kendinden önceki GGML (Georgi Gerganov Machine Learning) dosya formatının yerini

almıştır. GGML, başlangıçta yüksek performanslı tensor operasyonları için bir kütüphane ve model paylaşımı için erken bir formattı, ancak esneklik ve genişletilebilirlik açısından sınırlıydı. GGUF, bu sınırlamaları aşarak, modelin ağırlıklarını, önyargılarını ve ek meta verilerini (tokenizer sözlüğü, bağlam uzunluğu, tensör bilgileri) tek bir birleşik dosyada birleştirir. Bu konsolidasyon, modellerin hızlı yüklenmesini sağlar ve infaz sırasında verimliliği artırır. GGUF, FP32, FP16, bfloat16 ve 2-bit'ten 8-bit'e kadar geniş bir yelpazede nicemlenmiş tam sayı formatlarını destekler (18). Bu format, nicemlenmiş modellerin uç cihazlarda standart ve verimli bir şekilde dağıtılması için temel altyapıyı sunmaktadır.

4.2. whisper.cpp ve Donanım Hızlandırma Mekanizmaları

GGUF formatının infazı, genellikle GGML kütüphanesini temel alan hafif ve yüksek performanslı uygulamalar aracılığıyla gerçekleştirilir. **whisper.cpp**, OpenAI'nin Whisper modelinin, harici bağımlılıkları olmayan, saf C/C++ ile yazılmış yüksek performanslı bir infazıdır (20). Bu proje, düşük gecikmeli, CPU merkezli çıkarıma odaklanmaktadır.

whisper.cpp, temelinde GGML'yi kullanarak gerekli hesaplama grafiklerini yönetir ve yürütür. Gerçek zamanlı performans hedefine ulaşmak için, platforma özgü SIMD (Single Instruction Multiple Data) komut setlerini etkin bir şekilde kullanır:

1. **ARM NEON:** Raspberry Pi 5 veya Apple Silicon gibi ARM tabanlı mimarilerde, NEON intrinsikleri kullanarak hızlandırma sağlanır (20). NEON teknolojisi, ses ve video işleme, konuşma tanıma ve derin öğrenme algoritmalarını hızlandırmak için tasarlanmış bir mimari uzantısıdır. Bu, uç cihazlarda gecikmenin azaltılması için kritik bir faktördür (21).
2. **x86:** Benzer şekilde, x86 mimarileri için AVX intrinsikleri desteği mevcuttur.
3. **GPU Desteği:** whisper.cpp ve temelindeki GGML kütüphanesi, GPU hızlandırması için Metal, Vulkan ve NVIDIA gibi çeşitli arka uçları da destekleyerek esneklik sunar.

Bu C/C++ tabanlı optimizasyonlar ve donanım intrinsiklerinin kullanımı, nicemlenmiş modellerin düşük kaynaklı cihazlarda $RTF < 1.0$ hedefine ulaşmasını sağlayan teknik zemini oluşturur.

4.3. Mimari Zorluklar ve Streaming ASR Optimizasyonları

GGUF/GGML ekosistemi önemli performans kazanımları sağlasa da, dağıtım mimarisinin bütünlüğü ve güvenlik konuları da göz önünde bulundurulmalıdır. Yapılan incelemeler, GGML kütüphanesinin girdi dosyasını yetersiz doğrulaması nedeniyle yığın taşması (heap overflow) gibi bellek bozulması güvenlik açıklarına sahip olabileceğini göstermiştir (22). Güvenli ticari veya kurumsal dağıtımlar için, bu tür temel infaz kütüphanelerinin güvenlik denetimlerinin titizlikle yapılması gerekmektedir.

Ayrıca, düşük RTF hedefine ulaşmak için sadece nicemleme yeterli değildir; infaz stratejilerinin de gerçek zamanlı ses akışına göre optimize edilmesi gerekir. Geleneksel ASR'nin aksine, streaming ASR, infaz sırasında tam bağlam eksikliği nedeniyle daha zordur (23).

Transformatör tabanlı streaming modeller, genellikle zaman kısıtlı veya parça tabanlı (chunk-wise) dikkat mekanizmaları kullanır.

Literatürde önerilen gelişmiş yöntemler arasında, **Zaman Kaydırmalı Bağlamsal Dikkat (Time-Shifted Contextual Attention - TSCA)** ve **Dinamik Sağ Bağlam Maskeleyme (Dynamic Right Context Masking)** yer alır (24). TSCA, geleneksel parça tabanlı infazın, parçanın sonundaki karelerin yetersiz ileri bağlama sahip olması sorununu çözer. Mevcut parçanın son karelerini, bir sonraki parça için 'ileri bağlam' olarak yeniden konumlandırarak, gecikmeyi artırmadan daha doğru tahminler yapılmasını sağlar. Bu, nicemlenmiş GGUF modellerinin düşük gecikmeli canlı alt yazı hizmetleri için nasıl entegre edilebileceği konusunda önemli bir mimari yönelimdir.

5. Dil Spesifik Uygulamalar: Türkçe ASR Performansı ve İyileştirme Stratejileri

5.1. Whisper'ın Türkçe ASR Benchmarkları ve İhtiyaç Analizi

Whisper'ın çok dilli eğitimi, Türkçe ASR alanında önemli bir ilerleme sağlamıştır. Ancak, Türkçe gibi morfolojik açıdan zengin, aglütinatif ve düşük kaynaklı dillerde optimal doğruluk için ek adaptasyon gereklidir.

Mevcut Türkçe ASR Leaderboard verileri, genel Whisper modellerinin (örneğin **large-v,medium**) belirli Türkçe veri setlerinde %14 ile %17 arasında değişen Word Error Rate (WER) oranlarına sahip olduğunu göstermektedir (25). Bu WER seviyeleri, ticari kalitede uygulamalar için genellikle çok yüksektir. Leaderboard'lardaki gözlemler, ince ayarlı küçük Whisper varyantlarının bile , bazı temel büyük modellere karşı WER açısından rekabetçi, hatta bazı durumlarda daha iyi performans gösterdiğini ortaya koymuştur (25). Bu durum, model boyutundan ziyade dil özelinde yapılan ince ayarın (fine-tuning) kritik önemini vurgular.

5.2. İnce Ayar (Fine-Tuning) ile Performansın Dramatik İyileşmesi

Türkçe ses verileri üzerinde (örneğin Common Voice gibi) yapılan ince ayar çalışmaları, Whisper modellerinin performansında dramatik iyileşmeler olduğunu kanıtlamaktadır. Yayınlanan araştırmalar, Türkçe diline adapte edilmiş Whisper modelleri için %52.38'e varan WER düşüşleri gözlemlemiştir (26). Bu denli yüksek bir performans artışı, Türkçe için Whisper modelinin kullanıldığı ASR çözümlerinde ince ayarın bir opsiyon değil, doğruluğu kabul edilebilir seviyelere çekmek için zorunlu bir adım olduğunu göstermektedir.

Bu araştırma, ince ayarlı Whisper modellerinin Türkçe için sağlam ve doğru bir E2E ASR sistemi olarak başarıyla uyarlanabileceğini vurgulamaktadır.

5.3. Sinerjik Optimizasyon Stratejisi

Uç cihazlarda yüksek performanslı bir Türkçe ASR sistemi oluşturmak için izlenmesi gereken optimal strateji, ince ayar ve nicemlemenin sinerjik bir şekilde kullanılmasıdır.

1. **Doğruluk Önceliği:** Öncelikle, Whisper modelinin Türkçe veri setleri üzerinde ince ayarlanması gerekmektedir. Bu adım, modelin yüksek WER değerlerini düşürerek dilin fonetik ve morfolojik karmaşıklıklarına uyum sağlamasını garantiler.
2. **Uç Erişilebilirlik:** İnce ayarlanmış model, yüksek doğruluk sağlasa da, boyut ve gecikme açısından hala sınırlı kaynaklara sahip cihazlar için elverişsizdir. Bu noktada, Eğitim Sonrası Nicemleme (PTQ) uygulanarak modelin boyutu küçültülmeli ve infaz hızı artırılmalıdır. Dinamik **int8** veya **Q_{KM}** gibi agresif GGUF nicemleme tiplerinin kullanımıyla, modelin doğruluğu en az kayıpla korunurken, uç cihazlarda Real-Time Factor **1.0** 'ın altına çekilebilir.

Bu sinerjik yaklaşım, ticari uygulamalar için hem yüksek doğruluk (ince ayar sayesinde) hem de düşük gecikme (nicemleme ve **whisper.cpp** optimizasyonları sayesinde) gereksinimlerini aynı anda karşılar. Ayrıca, Gemma tabanlı Türkçe LLM'lerin bile GGUF formatında yayınlanması (27), GGUF'un sadece ASR değil, genel Türkçe makine öğrenimi uygulamalarında kabul görmüş ve güvenilir bir dağıtım formatı olduğunu teyit etmektedir.

6. Sonuç ve Önerilen Temel Yayın Listesi (Kaynakça)

6.1. Temel Çıkarımlarının Sentezi

Bu literatür taraması, OpenAI Whisper ASR modelinin uç nokta cihazlarda başarılı bir şekilde dağıtılması için nicemleme tekniklerinin ve özel infaz çerçevelerinin zorunlu olduğunu kanıtlamıştır.

1. **Nicemleme Zorunluluğu:** Whisper'ın büyük model yapısı, **RTF < 1.0** hedefine ulaşmayı engeller. Nicemleme (özellikle PTQ), model boyutunu %45-57 oranında azaltarak ve gecikmeyi düşürerek bu sorunu çözer.
2. **Optimal Nicemleme Şeması:** Dinamik Nicemleme, Whisper'ın Transformatör mimarisiyle uyumluluğu nedeniyle statik yöntemlere göre üstündür. **INT8** kesinliği, en iyi doğruluk/sıkıştırma dengesini sağlarken, daha agresif formatlar (**INT4 / INT3**) özellikle gürültülü koşullarda model sağlamlığını tehlikeye atmaktadır. Optimal doğruluk için **per-channel simetrik** şemalar (Quanto) tercih edilmelidir.
3. **Dağıtım Mimarisi:** Nicemlenmiş modellerin uç cihazlarda verimli infazı, GGUF formatı ve C/C++ tabanlı, ARM NEON/SIMD ile optimize edilmiş **whisper.cpp** infaz çerçevesi aracılığıyla sağlanır. Bu çerçeveler, donanım hızlandırma yoluyla gerçek zamanlı performansı mümkün kılar.
4. **Türkçe Spesifikasyonu:** Türkçe ASR uygulamalarında yüksek doğruluk elde etmek için Whisper'a özgü ince ayar kritik öneme sahiptir. Bu ince ayarın ardından uygulanan nicemleme, ticari düzeyde kullanıma uygun, optimize edilmiş ve dilbilgisel olarak doğru ASR sistemlerinin geliştirilmesinde sinerjik bir etki yaratır.

Alıntılanan çalışmalar

1. Frontier Research on Low-Resource Speech Recognition Technology - MDPI, erişim tarihi Aralık 15, 2025, <https://www.mdpi.com/1424-8220/23/22/9096>
2. Automatic Speech Recognition in the Modern Era: Architectures, Training, and Evaluation, erişim tarihi Aralık 15, 2025, <https://arxiv.org/html/2510.12827v1>
3. Robust Speech Recognition via Large-Scale Weak Supervision - ResearchGate, erişim tarihi Aralık 15, 2025, https://www.researchgate.net/publication/366136304_Robust_Speech_Recognition_via_Large-Scale_Weak_Supervision
4. Robust Speech Recognition via Large-Scale Weak Supervision, erişim tarihi Aralık 15, 2025, <https://proceedings.mlr.press/v202/radford23a.html>
5. openai/whisper: Robust Speech Recognition via Large-Scale Weak Supervision - GitHub, erişim tarihi Aralık 15, 2025, <https://github.com/openai/whisper>
6. arxiv.org, erişim tarihi Aralık 15, 2025, <https://arxiv.org/html/2503.09905v1>
7. Automatic Speech Recognition in the Modern Era: Architectures, Training, and Evaluation, erişim tarihi Aralık 15, 2025, <https://arxiv.org/html/2510.12827>
8. Performance and Efficiency Evaluation of ASR Inference on the Edge - MDPI, erişim tarihi Aralık 15, 2025, <https://www.mdpi.com/2071-1050/13/22/12392>
9. Paper page - Quantization for OpenAI's Whisper Models: A Comparative Analysis - Hugging Face, erişim tarihi Aralık 15, 2025, <https://huggingface.co/papers/2503.09905>
10. Quantizing Whisper-small: How design choices affect ASR performance - arXiv, erişim tarihi Aralık 15, 2025, <https://arxiv.org/html/2511.08093v1>
11. GenPTQ: Green Post-Training Quantization for Large-Scale ASR Models with Mixed-Precision Bit Allocation * - ACL Anthology, erişim tarihi Aralık 15, 2025, <https://aclanthology.org/2025.findings-emnlp.566.pdf>
12. [2511.08093] Quantizing Whisper-small: How design choices affect ASR performance, erişim tarihi Aralık 15, 2025, <https://arxiv.org/abs/2511.08093>
13. VOICE COMMAND RECOGNITION WITH DEEP NEURAL NETWORK ON EDGE DEVICES, erişim tarihi Aralık 15, 2025, https://hammer.purdue.edu/articles/thesis/VOICE_COMMAND_RECOGNITION_WITH_DEEP_NEURAL_NETWORK_ON_EDGE_DEVICES/15057192
14. Investigate alternative approach for Q4 quantization · Issue #397 · ggml-org/llama.cpp, erişim tarihi Aralık 15, 2025, <https://github.com/ggml-org/llama.cpp/issues/397>
15. Understanding GGML Files: A Deep Dive into Quantization and Visualization of File Structure - Abhik Sarkar, erişim tarihi Aralık 15, 2025, <https://www.abhik.xyz/articles/ggml-structure>
16. GGUF versus GGML - IBM, erişim tarihi Aralık 15, 2025, <https://www.ibm.com/think/topics/gguf-versus-ggml>

17. ggml/docs/gguf.md at master · ggml-org/ggml - GitHub, erişim tarihi Aralık 15, 2025, <https://github.com/ggml-org/ggml/blob/master/docs/gguf.md>
18. llama.cpp: The Ultimate Guide to Efficient LLM Inference and Applications - PyImageSearch, erişim tarihi Aralık 15, 2025, <https://pyimagesearch.com/2024/08/26/llama-cpp-the-ultimate-guide-to-efficient-llm-inference-and-applications/>
19. llama.cpp - Wikipedia, erişim tarihi Aralık 15, 2025, <https://en.wikipedia.org/wiki/Llama.cpp>
20. ggml-org/whisper.cpp: Port of OpenAI's Whisper model in C/C++ - GitHub, erişim tarihi Aralık 15, 2025, <https://github.com/ggml-org/whisper.cpp>
21. Neon - Arm Developer, erişim tarihi Aralık 15, 2025, <https://developer.arm.com/Architectures/Neon>
22. GGML GGUF File Format Vulnerabilities | Databricks Blog, erişim tarihi Aralık 15, 2025, <https://www.databricks.com/blog/ggml-gguf-file-format-vulnerabilities>
23. Simul-Whisper: Attention-Guided Streaming Whisper with Truncation Detection - arXiv, erişim tarihi Aralık 15, 2025, <https://arxiv.org/html/2406.10052v1>
24. [Literature Review] Improving Streaming Speech Recognition With Time-Shifted Contextual Attention And Dynamic Right Context Masking - Moonlight | AI Colleague for Research Papers, erişim tarihi Aralık 15, 2025, <https://www.themoonlight.io/en/review/improving-streaming-speech-recognition-with-time-shifted-contextual-attention-and-dynamic-right-context-masking>
25. Turkish Asr Leaderboard - a Hugging Face Space by ysdede, erişim tarihi Aralık 15, 2025, https://huggingface.co/spaces/ysdede/turkish_asr_leaderboard
26. Implementation of a Whisper Architecture-Based Turkish Automatic Speech Recognition (ASR) System and Evaluation of the Effect of Fine-Tuning with a Low-Rank Adaptation (LoRA) Adapter on Its Performance - MDPI, erişim tarihi Aralık 15, 2025, <https://www.mdpi.com/2079-9292/13/21/4227>
27. QuantFactory/wiroai-turkish-llm-9b-GGUF - Hugging Face, erişim tarihi Aralık 15, 2025, <https://huggingface.co/QuantFactory/wiroai-turkish-llm-9b-GGUF>
28. Arm SIMD: Optimize, Migrate, and Accelerate C/C++ for Peak Performance, erişim tarihi Aralık 15, 2025, <https://developer.arm.com/servers-and-cloud-computing/arm-simd>